SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

主办方 / Organizers：奇点智能研究院 Singularity Intelligence Research Institute　CSDN

合作伙伴 / Partners：CECW 中能拾贝　OMNI-AI　乐奇 Rokid　异步社区 www.epubit.com　清華大學出版社 TSINGHUA UNIVERSITY PRESS

2026技术大会活动预告

奇点智能研究院 Singularity Intelligence Research Institute CSDN

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

4月17-18日·上海 | 9月11-12日·北京

PM – Summit 2026

全球产品经理大会

Product Management Summit

7月17-18日·北京

CPP – Summit 2026

全球系统软件技术大会

C++ and System Software Summit

11月20-21日·上海

GOSIM®

GOSIM 2026

5月4-6日·巴黎 | 10月16-18日·深圳

2026年度 1024程序员日

智汇潮湃"人工智能+"创新大赛

AI构建世界 智能引领未来

10月24-25日·多城联动

媒体服务——多形态媒体矩阵

奇点智能研究院 Singularity Intelligence Research Institute　CSDN

为客户提供定制化内容服务，包括但不限于视频、文章、评测、专题报告、白皮书等多品类内容

访谈 → 新闻稿件 → 产品解读 → 产品评测 → 客户案例 → 活动报道 → 专题

直播栏目

《AI进化论》

《涛滔不绝》

《万有引力》

文章栏目

《AGI技术50人》

《近匠》

《开源英雄》

权威出版

《新程序员》

聚集全球技术大师、行业技术先锋，从前沿技术到行业先锋，为您带来全方位的深刻洞见和专业解读。

《专题报告》

《AI原生软件研发成熟度模型AISMM》
《2025全球开源发展报告》
云原生、AI赋能软件研发、数据库、操作系统等领域报告

技术咨询——企业信赖的策略伙伴

奇点智能研究院 Singularity Intelligence Research Institute CSDN

依托雄厚的专家团队和丰富的行业经验，通过12大技术咨询板块，提供涵盖人工智能、架构设计、产品管理等全栈技术咨询解决方案，帮助企业在数字化、智能化、万物互联的变革中打造关键创新能力，加速技术和产品迭代升级。

6大核心研究领域

01 智能化软件工程与研发效能

02 AI系统软件技术栈

03 企业开源软件设计与治理

04 软件架构设计与重构

05 系统软件性能工程与优化

06 智能化战略转型与落地

SITS 2026
奇点智能技术大会
Singularity Intelligence Technology Summit

演讲主题 Topic

Agent设计模式：从认知架构到工程落地

黄　佳

新加坡科技研究局(A*STAR)高性能计算研究所AI研究员
《Agent 设计模式》作者

奇点智能研究院 Singularity Intelligence Research Institute CSDN

SITS 2026 奇点智能技术大会 Singularity Intelligence Technology Summit

Agent 设计模式
从 认知架构 到 Harness 工程

黄佳 · A*STAR · 奇点智能技术大会 · 2026.04 上海

《Agent设计模式》作者 · Manning《Designing AI Agents》作者
奇点智能研究院 高级讲师

今天的目标

1

从混乱到系统

为何需要设计模式

2

双轴框架思想进化

从单轴认知到正交矩阵的演化

3

Harness 工程视角

设计模式和工程原理

4

双系统拆解

Claude Code × OpenClaw / Hermes

5

未来展望

Agent系统的未来

从设计模式框架到工程实战

AI 工程思维的三级跃迁

2023

Prompt Engineering

调教一句话
→ 模型就听话

你是一个……
请你一步一步的……

→

2024-2025

Context Engineering

调教上下文
→ 模型就懂项目

CLAUDE.md, AGENTS.md
SKILL.md

→

2026

Harness Engineering

调教整个系统
→ Agent 就可控

循环/工具/权限
上下文/成本

Harness = 缰绳。Agent 跑起来了，谁在拽缰绳？
模型是CPU / Harness是操作系统 / Agent是电脑

https://github.com/huangjia2019

黄佳

新加坡科研局AI 研究员 | 奇点智能研究院高级讲师 | 技术作者

主攻方向为大语言模型的开发和应用实践、AI in FinTech、AI in MedTech

专栏代表作：

- 《LangChain 实战课》 《MCP和A2A协议实战》
- 《RAG进阶训练营》《Claude Code工程化实战》

第一部分

从混乱到系统

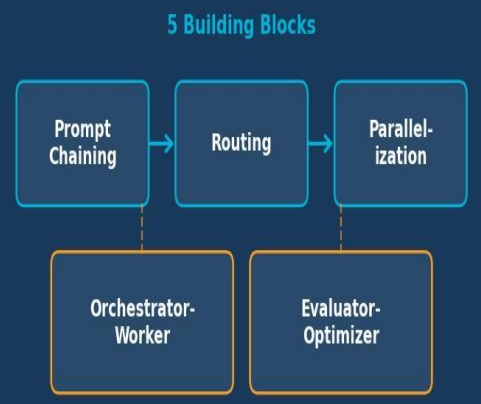
为什么 Agent 需要设计模式和 Harness？

01

Agent 架构的混乱时代

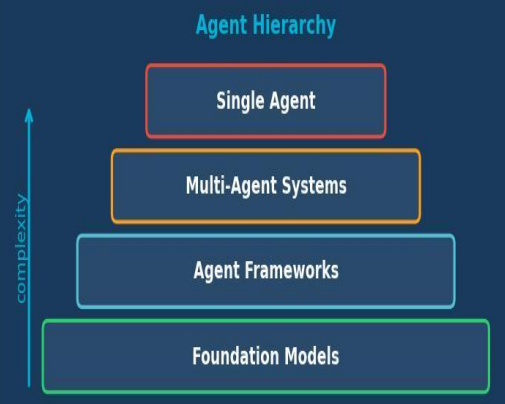
Anthropic

5种构建模块：
提示链、路由、并行化，
编排器-工作者、
评估-优化器



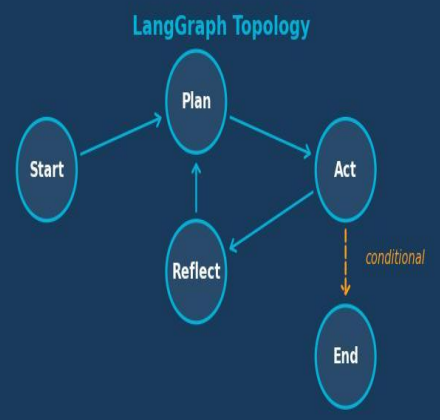
Google

Agent 分类：
单Agent、
多Agent、
Agent协作模式



LangChain

LangGraph 概念：
状态、 节点、边、
条件边、
监督者等一系列模式



OpenAI

Agents SDK：
交接、护栏、
追踪、工具使用、
运行器模式

问题：这些分类只覆盖了"数据怎么流动"（拓扑）
没有回答"Agent 需要什么能力"（认知）

我知道有问题，但不知道问题叫什么名字

「我的 Agent 总是胡说八道」

→ 你缺的是 **反思模式** - 生成-评审循环 (Generator-Critic）
或 经验复用（Experience Replay）

「多个 Agent 互相打架，越做越乱」

→ 你缺的是 **协作模式** - 层级委派（Hierarchical Delegation）
+ 交接链（Handoff Chain）

「对话长了就忘事」

→ 你缺的是 **记忆模式** - 分层保留 (Hierarchical Retention）
+ 进度追踪（Progress Tracking）

「同一个 prompt 时灵时不灵」

→ 你缺的是 **感知模式** - 上下文分诊（Context Triage）
以及 **推理模式** - 思维链 + 自一致性
（Chain-of-Thought + Self-Consistency）

共同的根因：

你不是缺能力
你是缺一张思维地图

每个痛苦都对应一种
已被命名的设计模式

你只是不知道它叫什么
以及它在地图的哪个位置

你需要一张地图 — 从痛苦到模式的导航系统

① 定位痛苦

「Agent 胡说八道」
是什么类型的问题？

→

② 找到坐标

认知轴 = 治理
拓扑轴 = 单体

→

③ 取出模式

Guardian 模式
+ Audit Trail 模式

这张地图 = 双轴正交框架

纵轴（认知轴）：你的 Agent 缺什么能力？

感知 · 记忆 · 推理 · 行动 · 反思 · 协作 · 治理

横轴（拓扑轴）：你的 Agent 数据怎么流动？

单体 · 管道 · 编排 · 层级 · 网络 · 涌现

交叉点 = 27 种已命名的设计模式 —— 你的问题一定在某个交叉点上

第二部分

双轴正交框架

Agent 设计的系统地图

02

第一轴：7种认知功能 — Agent 需要什么能力？

感知 Perception	记忆 Memory	推理 Reasoning	行动 Action	反思 Reflection	协作 Collaboration	治理 Governance
理解输入 上下文管理	存储和检索 知识管理	分析问题 制定策略	执行操作 工具调用	自我评估 迭代改进	多Agent协同 任务分配	安全约束 行为边界

三层架构：

核心认知（Core Cognition）：感知 → 记忆 → 推理 → 行动 —— "做事"

元认知（Meta-Cognition）：反思 —— "自省"

系统智能（System Intelligence）：协作 + 治理 —— "协调 + 约束"

治理 = Harness Engineering 在认知框架中的映射

Agent Capability Stack — 三层同心环

外层：系统智能

协作 — 多 Agent 协同

治理 — 行为边界约束

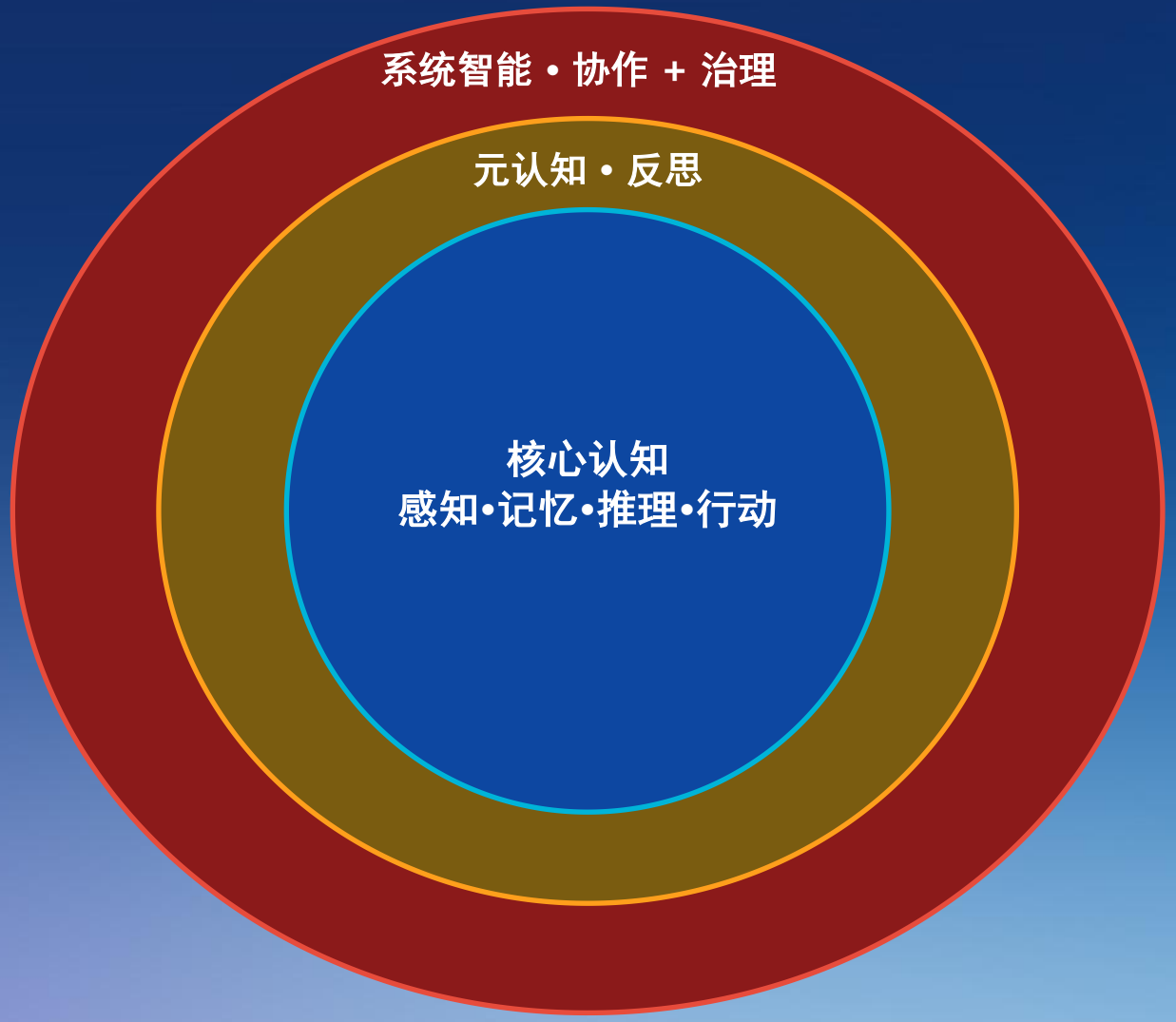
中层：元认知

反思 — 自我评估改进

内层：核心认知

感知 → 记忆 → 推理 → 行动

Agent 的核心处理链路



第二轴：6种执行拓扑 — 数据如何流动？

Chain 链式	Route 路由	Parallel 并行	Orchestrate 编排	Loop 循环	Hierarchy 层级
→ → →	⟨ ⟩	⇉ ⇉	✦	↻	▽
A → B → C 顺序处理	条件分发 选一条路径	并发扇出 同时多路	中央协调 动态调度	迭代直至 收敛	多级委派 树状结构

与 Anthropic 的 5 种 构建模块 高度吻合

Chain ≈ 提示链 | Route ≈ 路由 | Parallel ≈ 并行化
Orchestrate ≈ 编排-工作者 | Loop ≈ 评估-优化器

我增加了 Hierarchy（多级委派）— CrewAI / MetaGPT

从设计模式到双轴矩阵 — 思想进化

第一代：认知维度列表（2025）

《Agent设计模式》中文版

6维认知：感知·记忆·推理·行动·反思·协作
21种模式，单轴分类

解决了"Agent需要什么能力"的问题

→

第二代：双轴正交矩阵（2025-2026）

《Designing AI Agents》Manning 英文版

7维认知（+治理）× 6种拓扑
27种模式，双轴定位

解决了"能力用什么拓扑实现"的问题

从单轴到双轴 = 从分类目录到设计坐标系

27种 Agent 设计模式 — 双轴正交矩阵

认知功能	链式 Chain	路由 Route	并行 Parallel	编排 Orchestrate	循环 Loop	层级 Hierarchy
感知 Perception	语义压缩	上下文分诊	多模态融合	渐进式发现		
记忆 Memory	RAG 管道	层级保留		进度追踪	失败日志	
推理 Reasoning	思维链	复杂度路由	并行探索		迭代假设检验	
行动 Action	提示链	工具分发		计划-执行		护栏三明治
反思 Reflection		技能包			生成-评判	经验回放
协作 Collaboration	交接链		扇出/汇聚		对抗审查	层级委派
治理 Governance		审批门控	渐进承诺	可观测性 Harness		爆炸半径控制

如何解读 <Agent 设计模式> 矩阵

空白不是遗漏 — 而是设计洞察

不是所有认知功能都需要每种拓扑

42个格子
27个模式
15个空白

关键例子："同一拓扑，不同认知目标"

Orchestrate 编排拓扑的三种用法：

- ✦ 渐进式发现（感知）— 动态发现上下文
- ✦ 计划-执行（行动）— 规划后执行
- ✦ 可观测性 Harness（治理）— 可观测性编排

→ 这就是为什么需要两个轴

单看拓扑 = 只知道数据怎么流 | 加上认知维度 = 知道为什么这样流

AI Agent 设计的结构化框架

问题：「Agent」被滥用，领域一片混沌

轴1：7种认知功能 — Agent 需要思考和做什么？

- 感知 — 处理输入
- 记忆 — 存储检索
- 推理 — 逻辑推断
- 规划 — 任务分解
- 行动 — 工具调用
- 反思 — 自我评估
- 治理 — 安全约束

→

① 定义认知需求（轴1）
↓
② 映射设计模式（轴2）
↓
③ 架构决策点
↓
④ 生成目标架构

方案：系统化、解耦的架构

轴2：27种命名设计模式

单 Agent 模式

- ReAct
- 思维链
- 计划-执行
- 生成-评判
- RAG 管道
- 经验回放

组合与编排

- 监督者-工作者
- 自主流水线
- 对抗审查
- 层级委派

参考：《Designing AI Agents》黄佳 · Manning Publications · 2026

认知需求（轴1）决定 WHAT • 执行拓扑（轴2）决定 HOW • Harness Engineering 决定可控性

设计驱动的集成流程

为什么加"治理"维度？

2024年的6维框架没有治理——因为Agent还在实验室
2025年Agent进入生产环境 → 失控事件频发

无限循环烧token · Agent执行rm -rf · 输出敏感信息

治理不是可选的——它是生产级Agent的生死线

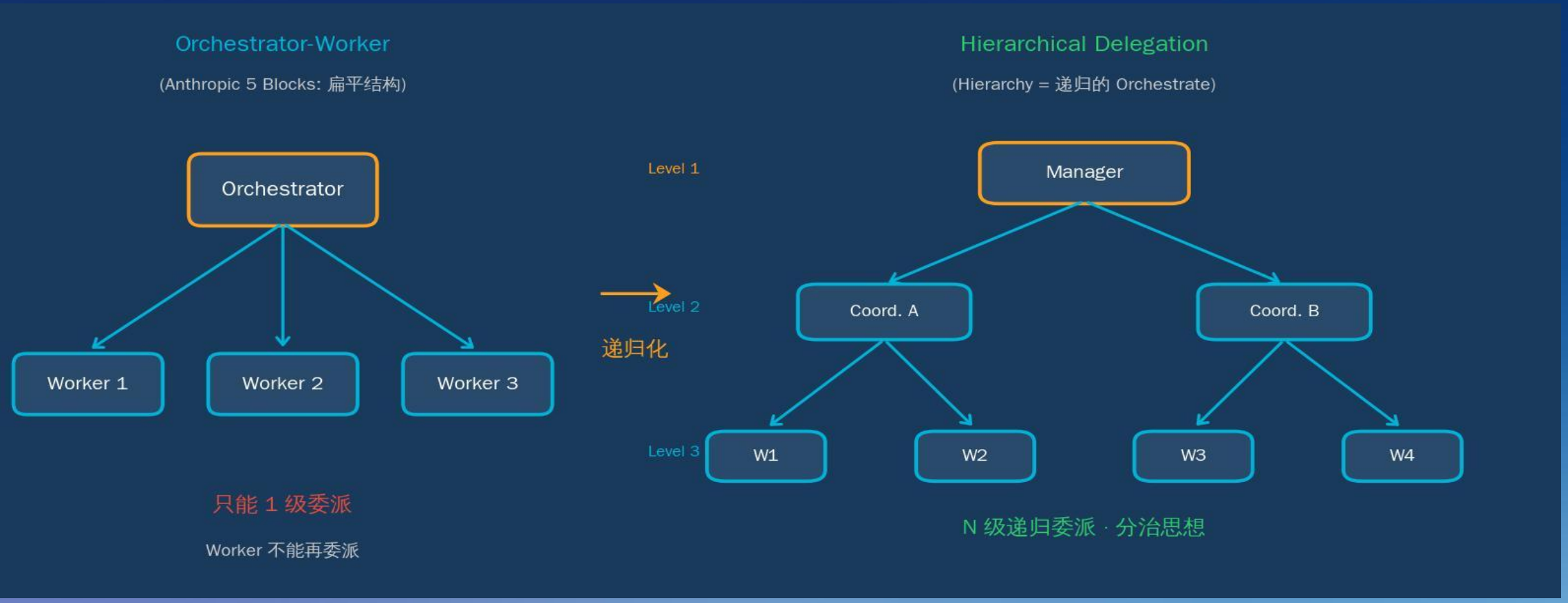
治理 = Harness Engineering 在认知框架中的映射

审批门控 · 爆炸半径控制
渐进承诺 · 可观测性 Harness

4种治理模式 → 全部对应 Harness 的权限/安全/可观测组件

为什么加"层级"拓扑？

虽然 Anthropic 的5种 构建模块 没有 Hierarchy 但现实中多级委派无处不在



Hierarchy = 递归的 Orchestrate · 分治思想在 Agent 世界的工程化表达

现实案例：Claude Code (Agent → SubAgent) · CrewAI (Manager → Worker → Sub-Worker) · MetaGPT (Architect → Engineer → Tester) · OpenClaw (Gateway → Agent → SubAgent)

第三部分

Harness 工程视角

模式如何被工程化？

03

AI 工程思维的三级跃迁

2023

Prompt Engineering

调教一句话
→ 模型就听话

你是一个……
请你一步一步的……

→

2024-2025

Context Engineering

调教上下文
→ 模型就懂项目

CLAUDE.md, AGENTS.md
SKILL.md

→

2026

Harness Engineering

调教整个系统
→ Agent 就可控

循环/工具/权限
上下文/成本

Harness = 缰绳。Agent 跑起来了，谁在拽缰绳？

铁证：Harness 比模型更重要

同一模型，换 Harness → +13.7 分

LangChain Deep Agents（2026.03）· 同模型 · 同 Benchmark · 仅调 Harness

Claude Code = $10 亿年化收入 = 一个 Harness 产品的价值

来源	关键观点
OpenAI (2025)	"Scaffolding matters more than the model itself"
Anthropic (2025)	"Agent performance is dominated by harness design"
LangChain (2026.03)	同模型 +13.7 分，仅改 Harness
Martin Fowler (2026)	"The agent is the system, not the model"

模式 → Harness 组件映射

Agentic Loop	→	生成-评判
工具系统	→	技能包、工具分发
上下文管理	→	上下文分诊、层级保留
权限控制	→	审批门控、爆炸半径控制
状态持久化	→	进度追踪、失败日志
事件钩子	→	护栏三明治

工程原理 ①：上下文窗口隔离

主会话

36.5K tokens 噪声

历史对话 + 无关上下文
信噪比低 → 准确率下降

子智能体

14K tokens 干净上下文

仅任务相关上下文
信噪比高 → 准确率提升

Sub-Agent 的价值 ≠ 更强的模型，= 更干净的上下文

分布式系统有进程隔离，Agent 有上下文隔离

工程原理 ②：声明式配置优先

声明式 = 可审计 + 可版本控制 + 可 AI 理解

Claude Code

CLAUDE.md
1 文件 All-in-One

DeepAgents

AGENTS.md
+ 中间件注入

OpenClaw

6 文件分离
SOUL/USER/TOOLS...

声明式是所有 Agent 配置系统的元原则

工程原理 ③：权限最小化

Read → 自动执行（低风险）

Edit → 需确认（中风险）

rm -rf → 永远禁止（高风险）

Claude Code

permission_mode
+ Hooks

DeepAgents

HITL 中间件
+ 路径白名单

OpenClaw

Per-Agent 白名单
+ DM Pairing

Harness 设计七原则（速记卡）

1	少即是多	— 工具 ≤ 25，组合涌现
2	循环有界	— 永远设 max_turns
3	权限最小	— 默认 deny，按需 allow
4	上下文必重注入	— 规范文件压缩后重注入
5	可观测优先	— 每轮记录工具+token+耗时
6	审核不可跳过	— 面向用户输出必过质量关卡
7	成本可感知	— 用户能看到每次花了多少钱

第四部分

双案例深度拆解

Claude Code vs OpenClaw

04

为什么选这两个系统？

Claude Code — 纵深型 Harness

单人编码 • 极简 • $10亿ARR

单线程 agentic loop
+ 15 工具
+ 少即是多

OpenClaw — 广度型 Harness

多人服务 • 全栈 • 13+渠道

Gateway + 绑定路由器
+ 6 文件声明式配置
+ 多 Provider 容错

同一范式（Harness），不同设计选择 → 验证模式的通用性

Claude Code ① — 感知与记忆

感知：上下文分诊（路由）

5级 CLAUDE.md 层次结构：

企业级 → 项目级 → 会话级 → 用户级 → 记忆级

技能库 / 规则库

工作原理：

根据当前任务，路由选择相关上下文

自动压缩长对话（~92% 压缩率）

模式坐标：感知 × 路由

记忆：层级保留（层级）

6层记忆体系：

① 工作记忆（当前对话）
② 会话记忆（会话内持久）
③ 项目记忆（CLAUDE.md）
④ 用户记忆（~/.claude/）
⑤ 企业记忆（组织级配置）
⑥ 外部记忆（MCP 工具）

模式坐标：记忆 × 层级

Claude Code ② — 反思与治理

反思：生成-评判（循环）

edit-test-fix 循环：

① 生成代码修改
② 运行测试
③ 分析失败原因
④ 迭代修复直至通过

这是一个经典的 Loop 拓扑
生成器和评判器在同一 Agent 内循环

模式坐标：反思 × 循环

治理：审批门控（路由）

PreToolUse Hooks + 权限模型：

执行工具前 → 检查权限
危险操作 → 人类审批
安全操作 → 自动放行

三级权限：

Allow → 自动执行
Ask → 需要确认
Deny → 禁止执行

模式坐标：治理 × 路由

Claude Code 架构全景 — 单线程 Harness 的极致

Agentic Loop

while (response.has_tool_calls):

① 组装上下文
prompt + history + 15 tools
② Claude API 推理
→ 文本 + tool_calls
③ 权限检查
allow / deny / ask
④ 执行工具 → 收集结果
⑤ 结果回注对话历史
⑥ h2A 双缓冲：用户实时干预

终止：end_turn 或 max_turns

核心原则：
简单循环 + 纪律工具 = 可控自主性

少而精：没有重构工具/测试工具 — 全靠组合涌现

六大 Harness 组件

Agentic Loop	推理→工具→回注→继续
15 个工具	Read/Write/Edit/Bash/Grep/Glob/Agent...
上下文管理	92% 压缩 + CLAUDE.md 重注入
权限控制	permission_mode + PreToolUse Hooks
Sub-Agent	worktree 隔离 · 递归委派
TodoWrite	任务分解 + 进度追踪

OpenClaw ① — 路由与记忆

路由：绑定路由器 → 审批门控

治理 × 路由

确定性匹配（非语义路由）

优先级规则：

Peer > Guild > Team >
Account > Channel > Default

可审计 · 可热加载 · 无需重启

记忆：双层架构 → 层级保留

记忆 × 层级

日志层：

memory/YYYY-MM-DD.md
自动加载近 2 天

持久层：

MEMORY.md（手动沉淀）

向量检索：BM25 + 语义混合

模式坐标：治理×路由 + 记忆×层级

OpenClaw ② — 隔离与可靠性

隔离：四级 Session → 爆炸半径控制

治理 × 层级

main：所有 DM 共享
省 token，有隐私风险

per-peer：同一人跨渠道合并

per-channel-peer：按渠道隔离
（推荐）

per-account-peer：最高隔离

可靠性：两阶段故障转移

第一阶段：
同 Provider 轮换 Auth Profile

第二阶段：
跨 Provider 降级
Claude → GPT → Gemini

指数退避：
认证 1-60min / 账单 5-24hr

7×24 服务场景的刚需 — 编码 Agent 挂了能等，Bot 挂了是事故

OpenClaw 架构全景 — 多渠道 Harness 的全栈设计

Gateway — 心脏（Mediator 模式）

WebSocket 控制面 · 永驻进程 · ws://127.0.0.1:18789
三帧协议：req / res / event
两阶段确认：防止 20 秒消息超时
绑定路由器：确定性路由（非语义匹配）
优先级：Peer > Guild > Team > Account > Channel

13+ 渠道适配器

Telegram	WhatsApp	Discord	Slack
Signal	iMessage	WeChat	

每个适配器：标准化消息格式 + 平台特有认证

6 文件声明式配置

AGENTS.md	角色定义
SOUL.md	人格/不变规则
TOOLS.md	工具使用指南
USER.md	用户画像
HEARTBEAT.md	定期行为
MEMORY.md	长期记忆

四级 Session 隔离

main	所有 DM 共享（省 token）
per-peer	同一人跨渠道合并
per-channel	按渠道隔离（推荐）
per-account	最高隔离

两阶段模型故障转移

Stage 1:
同 Provider 轮换 Auth Profile
Stage 2:
跨 Provider 降级
Claude → GPT → Gemini

指数退避：认证 1-60min
账单 5-24hr

6种核心模式 × 双系统映射

模式	Claude Code	OpenClaw
层级保留	CLAUDE.md 压缩+重注入	SOUL.md + MEMORY.md + 向量检索
生成-评判	edit-test-fix 循环	多 Agent 审核
审批门控	Hooks (PreToolUse)	DM Pairing + 工具白名单
技能包	SKILL.md 声明式	SKILL.md + 门控条件
上下文分诊	92% 自动压缩	滑动窗口 + 每日日志
爆炸半径控制	permission_mode	四级 Session + Docker 沙箱

模式不变，Harness 实现随场景变化

核心洞察

模式是跨平台通用的

同一个 生成-评判 模式：

Claude Code: edit → test → fix 循环（单 Agent 内）

OpenClaw: 多 Agent 交叉审核（跨 Agent）

模式不变，场景在变 • Harness 实现在变

从"模式不变"到"Harness不同" — 完整图景

设计模式不是行为处方

它是一种约束——让好的运行时行为更可能涌现的约束

你不能控制 Agent 的决策

你只能工程化它做决策的环境

设计模式 = 环境的蓝图 · Harness = 环境的实现

第五部分

总结与展望

05

Claude Managed Agents · Harness 即服务

Anthropic 2026-04-08 公测 · 把 Claude Code 运行时打包成云 API

三虚拟化组件 · 完全解耦 · 各自独立伸缩

Session
append-only 事件日志

- 持久化数小时
- Harness 崩溃也不丢
- wake(sessionId) 重新挂载

⇄

Harness
无状态 Agent Loop 编排器

- 上下文管理 · 压缩 · 缓存
- 水平伸缩 (cattle not pets)
- 多实例可挂同一 Session

⇄

Sandbox
隔离的工具执行环境

- 一次性容器 · 出错即重建
- Bash / Read / Write / Web
- 凭证永不进入 Sandbox

4 个核心 API · 一切皆资源

`POST /agents`
模型+工具+skills+MCP

`POST /environments`
容器模板 · secrets

`POST /sessions`
Agent + Env · 运行实例

`GET /v1/sessions/{id}/events`
SSE 流回

2026 Q2 · Harness 的三条演进路径

本地极致 · 云端托管 · 多渠道全栈 · 同一范式 · 三种哲学 · 不是替代

维度	Claude Code	Managed Agents	OpenClaw
发布时间	2024 起	2026.04.08 公测	2025 起
运营方	用户自管	Anthropic	用户自管
运行位置	用户机器 · 本地 CLI	Anthropic 云	自建服务器
模型	Claude 锁定	Claude 锁定	多模型可切换
核心架构	单线程 Agentic Loop	三虚拟化 · 水平伸缩	Gateway Mediator
工具	15 工具 × 5 原子	agent_toolset + MCP	13+ 渠道适配器
上下文	92% 压缩 + CLAUDE.md	Session 独立持久化	4 级 Session 隔离
许可证	MIT	商用 · 订阅	Apache 2.0
适用场景	开发者个人工具	企业级 Agent 后端	面向终端用户产品

个人开发 → Claude Code • 企业后端 → Managed Agents • 面向用户产品 → OpenClaw

Hermes Agent — 自演化 Harness 的开源先行者

Nous Research · MIT 开源 · 37K Stars · "The agent that grows with you"

① 程序性记忆

Procedural Memory

完成 5+ 工具调用的
复杂任务后
自动提取为 SKILL.md

下次同类问题
8 轮 → 3 轮
成本降 60%

② 冻结快照

Frozen Snapshot

会话开始时冻结上下文
Mid-session 写入磁盘
但不改 system prompt

Prompt Cache 命中率
大幅提升
成本降 60-80%

③ GEPA 自进化

Self-Evolution

进化算法自动优化
Skill 的 prompt
读 traces 理解失败原因

ICLR 2026 Oral
自动改进 Skill
人类只需 review

Claude Code 的 Harness 是人配置的，Hermes 的 Harness 能自我进化

工程师的角色：今天你写 SKILL.md → 将来你 review Agent 自动生成的 SKILL.md

从 Harness Engineering 到 Harness Evolution

2023	2024	2026	2028?
Prompt Engineering 调教一句话	Context Engineering 调教上下文	Harness Engineering 调教整个系统	Harness Evolution Agent 调教自己

每一代解锁的"可调教对象"不同

可调教对象	Prompt 文本	上下文窗口	循环/工具/权限/成本	约束本身
调教者	人类	人类	人类 + 工具	Agent（在人类监督下）
反馈环路	人工试错	人工 + 检索	自动化测试 + 指标	自动化 + 自我评估
失败代价	输出不好	幻觉/遗忘	失控/超支	约束漂移/自指崩溃

开放问题：Harness Evolution 是否可能？如果可能，不变层的边界在哪里？

"你不能设计一个能完全重写自己设计规则的系统 —— 但你可以设计一个能在固定规则下越来越聪明地运用规则的系统"

设计模式 + Harness = 完整图景

模式 = WHAT • Harness = HOW

27 种设计模式

Agent 需要什么能力
× 什么拓扑

（分类工具）

Harness Engineering

循环/工具/上下文
权限/成本

（落地方法）

三条工程原理

隔离/声明式
权限最小化

（设计准则）

GoF 23 模式改变了 OOP • 27 模式 + Harness 定义 Agent 工程

Agent 设计模式的未来：从工具到基础设施

2024-2025 • 模式发现期

单 Agent 模式成熟
ReAct + RAG 成为事实标准
框架百花齐放，标准缺失

2026-2027 • 模式工程期

双轴框架 → 系统化选型
Harness Engineering 成为新学科
MCP / A2A 协议层统一

2028+ • 模式基础设施期

Agent 设计模式 = 软件工程标配
治理从可选变为强制
自主系统进入关键基础设施

三个预判

① Agent 设计模式将成为软件工程师的必修课

GoF 23 种模式定义了面向对象 · 27 种 Agent 模式将定义智能体工程

② Harness 将取代模型成为核心竞争力

同一模型，不同 Harness → 13.7 分差距 · 调教系统 > 选模型

③ 治理将从"以后再说"变成"第一天就建"

88% 的 Agent 安全事件源于治理缺失 · 治理 = 生产级 Agent 的入场券

我们正站在 Agent 工程学的起点 —— 设计模式是这门学科的第一块基石

2023-2024 年我们学会了调教 Prompt
2025 年学会了调教上下文
2026 年该学调教整个系统了

Harness 是你的工作台
设计模式是你的工具箱

臣之所好者，道也
何时约束，何时自由
何时信任，何时控制
如庖丁之刀，游刃有余

— 黄佳

奇点智能研究院 Singularity Intelligence Research Institute CSDN

SITS 2026

奇点智能技术大会

Singularity Intelligence Technology Summit

感谢聆听

黄佳 · A*STAR